

Code Review Example: 1

```
#define CDMA_SUBSCRIPTION_SOURCE_NV 0
#define CDMA_SUBSCRIPTION_SOURCE_RUIM 1
```

80
81
82

David Turner

Sep 13, 2010

the functions below operate on an AModem object, so should be named amodem_switch_technology(),
amodem_set_cdma_xxxx

Reply ...

Reply 'Done'

Jaime A Lopez-Sollano

Sep 23, 2010

Done

Reply ...

Reply 'Done'

```
static const char* switchTechnology(AModem modem, AModemTech newtech, int32_t newpreferred);
static int set_cdma_subscription_source( AModem modem, ACdmaSubscriptionSource ss);
static int set_cdma_prl_version( AModem modem, int prlVersion);
```

83
84
85
86

Style Inconsistency!

Code Review Example: 2

```
if (noMedia) {
```

```
// in case the file is known and now is under a  
// .nomedia folder mark as not seen in order to  
// be removed from files table in the post scan.
```

```
if (entry != null && noMedia) {
```

478

479

480

481

482

Emilio López

Jan 15, 2012

Isn't '&& noMedia' redundant? noMedia can only be true inside the if block.

Reply ...

Reply 'Done'

David Marques

Jan 16, 2012

Thanks Emilio! My bad :D

Reply ...

Reply 'Done'

```
entry.mSeenInFileSystem = false;
```

```
}  
return null;
```

```
}
```

483

484

485

486

487

Silly mistake!

Code Review example: 3

#13244

```
62 >> if ((nbytes=recvfrom(fd,msgbuf,MSGBUFSIZE,0,
63 >> >> (struct sockaddr *) &addr,&
64 >> perror("recvfrom");
65 >> exit(1);
66 >> }
67 >> puts(msgbuf);
```

Michael Blanton

Feb 8, 2010

Isn't there a risk of msgbuf not being terminated? Reading all the way to MSGBUFSIZE seems to risk overwriting the implicit init done, or does the kernel always null-terminate the buffer, even if it is larger than MSGBUFSIZE?

I guess the worst case is probably not part of the spec.

Reply

Done

David Haddock

Feb 9, 2010

You are absolutely right. I will change it to write() the number of bytes received. I only tested it with the other file (sender.c) which sends "Hello, world" with the null-byte attached. I will send a patch when I get home. Thanks for the review!

Code Review Example: 4

```
54 »      »      if ( $this->isPermalink ) {  
55 »      »      »      $html .= Linker::link(  
56 »      »      »      »      »      »      »      SpecialPage::getTitleFor( 'ArticleFeedbackv5', $record[0]->p  
age_title ),  
57 »      »      »      »      »      »      »      wfMessage( 'articlefeedbackv5-special-goback' )->text());
```

Catrope

Apr 17, 2012

And this unfixes a security bug that was fixed earlier: this must be `->escaped()`, not `->text()`, because the second param to `link()` is HTML.

[Reply ...](#)

[Reply 'Done'](#)

Reha Sterbin

Apr 17, 2012

Done

[Reply ...](#)

[Reply 'Done'](#)

XSS Vulnerability!

Proofreading

You might be an Internet 'solopreneur', a lone wolf who writes, edits, formats, checks and publishes your own content. Or you might work for a creator or as a production editor, sub-editor or editor tasked to edit/proofread work produced by your colleagues.

Not you? Maybe you want to self-publish a book or thesis, perhaps error-check a company report? Maybe newsletters, business cards, product packaging or sign email, posting a Tweet or updating Facebook? They typed recently correct? Are you certain? It's not a

```
private static boolean needsXhtmlAttributes() {  
    try {  
        StringWriter writer = new StringWriter();  
        TransformerHandler probe = new TransformerHandler();  
        probe.setResult(new StringWriter());  
        probe.startDocument();  
        probe.startPrefixMapping("uri", "http://www.w3.org/1999/xhtml");  
        probe.endPrefixMapping();  
        probe.endDocument();  
        return writer.toString().contains("uri");  
    } catch (Exception e) {  
        throw new UnsupportedOperationException();  
    }  
}
```



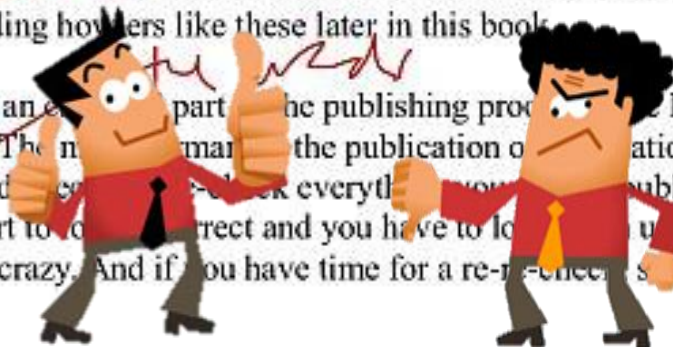
You might not have a proofreading process. Or you might be already be actively proofreading and want to get better at it.

"T
pe
int

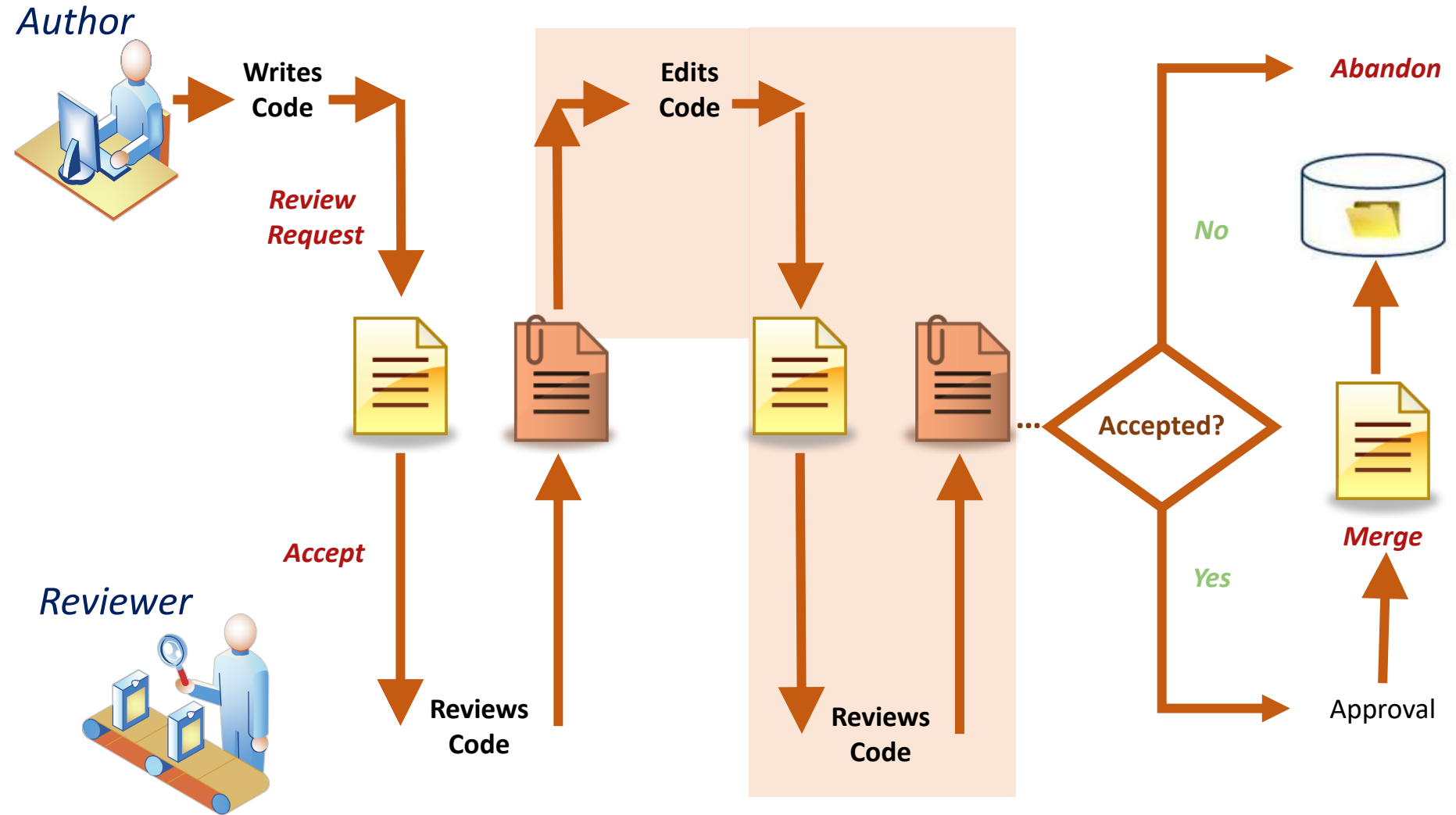
And forgotten

If you publish or print words of any kind, you won't want any mistakes to slip through. You want to ensure that what you produce is the best that it can be. This is especially true if you are printing anything. Mistakes on the web can be corrected. You don't have that luxury with print. Print has permanency. Just think how you'd feel if you'd been the sign writer who painted 'SHCOOL' in big white letters on the road when you meant to spell 'SCHOOL'. Or that you'd given the OK to a sign that read 'DRIVE-THRU ENTERANCE'. These are real examples. You'll find more proofreading howlers like these later in this book.

Again, proofreading is an essential part of the publishing process. The larger the font, the more you should check it. The more important the publication or the more the publication is for you, the more you should check it. You should check everything you publish until your eyes ache or until the words start to look incorrect and you have to look at it again just to make sure that you're not going crazy. And if you have time for a re-check, so much the better.

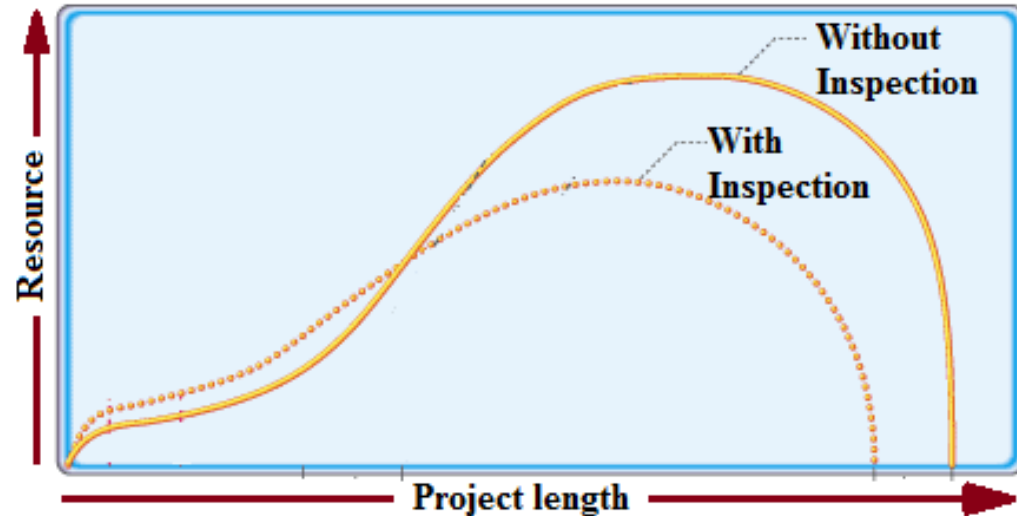


Code Review Workflow



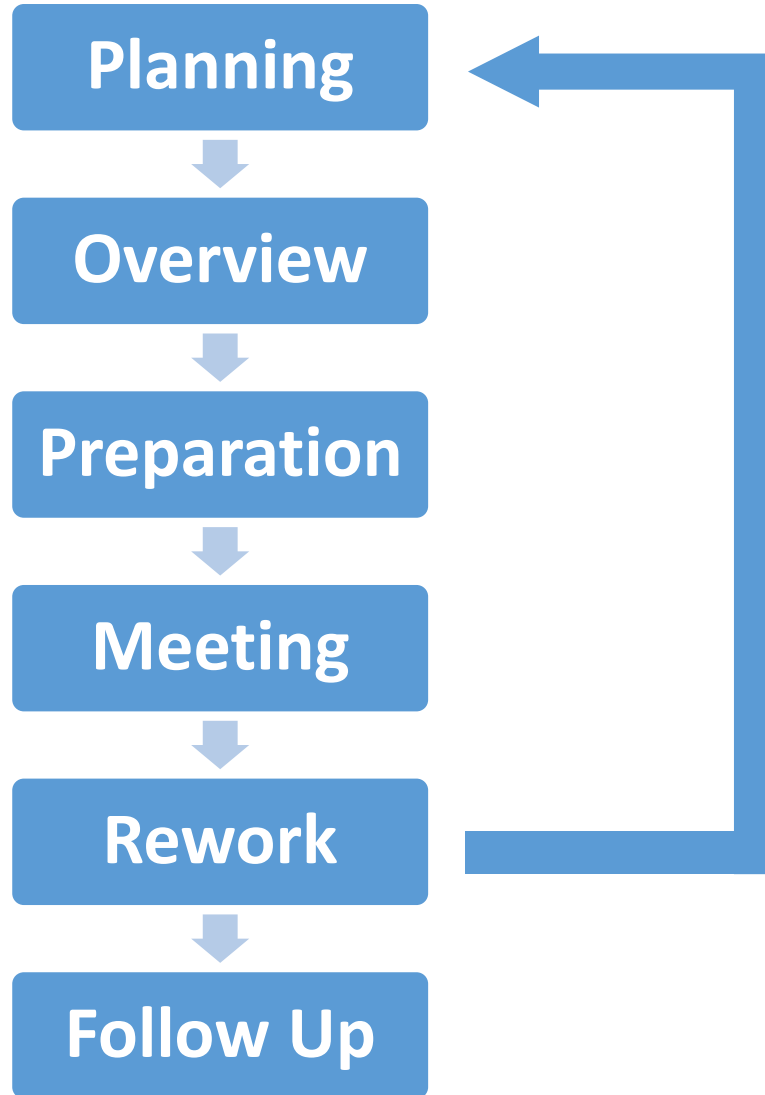
History

- Code Inspection
 - A.k.a. Fagan Inspection
 - Improves software quality



Fagan's "snail curve" (Advances in Software Inspection, Fagan 2001), image (adapted color version) taken from
(<http://www.embedded.com/design/prototyping-and-development/4023337/Verification-and-Validation>)

Software Inspection



- Problems
 - Process heavy
 - Arranging meetings difficult
 - High cost
 - Low adoption

Code Review Tools



Gerrit: <https://code.google.com/p/gerrit/>



Review Board: <https://www.reviewboard.org/>



Phabricator: <https://phabricator.org/>



Crucible:
<https://www.atlassian.com/software/crucible>

Code Review Efforts



Avg. 6 hours / week

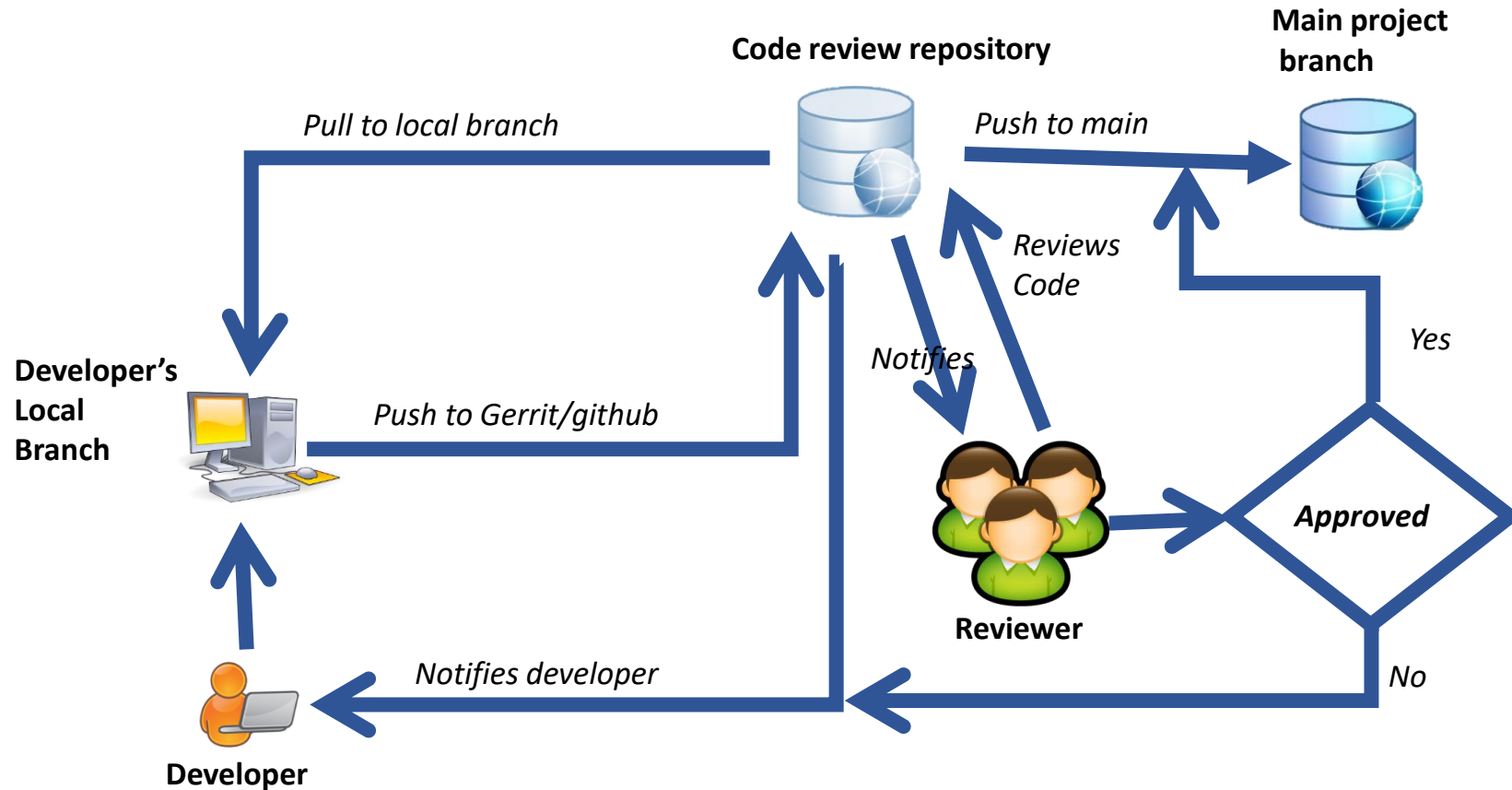


10%~15% of developer times

 **Microsoft** : 50,000 developers X 6 hours/ week X \$50 /hour

= \$15,000,000 / Week

Simplified code review workflow



Issues identified during code reviews

- ✓ Coding style
- ✓ Redundant checks
- ✓ Project design violation
- ✓ Misunderstood requirements
- ✓ Critical security defects
- ✓ Malicious code
- ✓ Inadequate input validation
- ✓ Unsafe methods
- ✓ Lack of exception handling

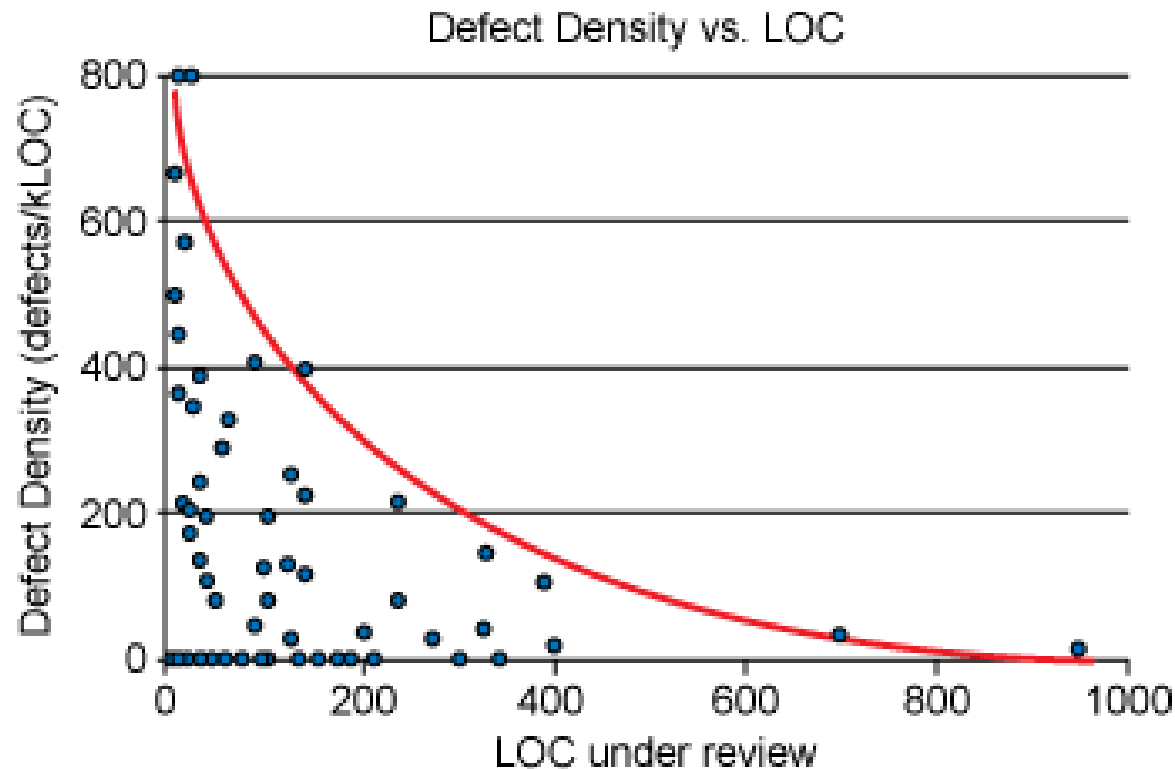
Issues identified during code reviews

- ✓ Coding style
- ✓ Redundant checks
- ✓ Project design violation
- ✓ Misunderstood requirements
- ✓ Critical security defects
- ✓ Malicious code
- ✓ Inadequate input validation
- ✓ Unsafe methods
- ✓ Lack of exception handling

Best practices for code reviews

Source: <https://smartbear.com/learn/code-review/best-practices-for-peer-code-review/>

1. Review fewer than 200–400 lines of code at a time



Source: <https://smartbear.com/learn/code-review/best-practices-for-peer-code-review/>

Be sure that authors annotate source code before the review begins.

- Annotations guide the reviewer through the changes
- Showing which files to look at first and defending the reason behind each code modification.
- Annotations should be directed at other reviewers to ease the process and provide more depth in context.

```
// Only valid in browsers that implement the HTML5 navigation
// pushState and accompanied APIs. When change to the navigation
// state is recognized then the page attempts to load the requested page.
_window.on('popstate', function (event) {

    this._popStateEventCount++;

    // Only process if the event fires as the result of the
    // back or forward button being clicked - skip if
    // the event is being raised on the page load.

    // This skips processing for WebKit browsers the first
    // time the event is raised.
    if ($.browser.webkit && this._popStateEventCount == 1) {
        return;
    }
}
```

Establish quantifiable goals for code review and capture metrics so you can improve your processes.

- Set a few tangible goals
 - reduce support calls by 15%,
 - cut the percentage of defects injected by development in half.
- Collect process metrics
 - **Inspection rate:** the speed with which a review is performed
 - **Defect rate:** the number of bugs found per hour of review
 - **Defect density:** the average number of bugs found per line of code

Goals



Use checklists, because they substantially improve results for both authors and reviewers.

- Developers often make the same mistakes over and over
- Omissions in particular are the hardest defects to find because it's difficult to review something that isn't there.
- Checklists are the most effective way to eliminate frequently made errors and to combat the challenges of omission finding.

Code Review CheckList		
Developer		
Name:		
E#:		
Reviewer		
Name:		
E#:		
Sl. No.	Check?	Remarks
Reviewing Comments and Coding Conventions		
1	Does the code respect the project coding conventions?	
2	Are the variable declarations properly commented?	
3	Are all functions, methods and classes documented?	
4	Are the functions/methods/classes really doing what the documentation says?	
5	Are function parameters used for input and output identified and documented in the function header?	
6	Are complex algorithms and code optimizations adequately commented?	
7	Does Commented out code have an explanation?	
8	Are comments used to show missing or yet to be implemented functionality?	
9	Is the modification Log upto Date?	
10	Are the comment blocks consistent and of same size?	
Reviewing Error Handling		
1	Are errors properly handled each time the function returns?	
2	Are error messages conveying what exactly is the error that has occurred?	
3	Are resources and memory released in all error paths?	

Verify that the defects are actually fixed.

- How will the bugs be fixed?
- Many teams do not have a systematic method for *fixing* the bugs they've worked so hard to find.
- Reviewers should log bugs, discuss them with the author, and approve changes in the code.



Foster a good code review culture in which finding defects is viewed positively.

- Peer review can put strain on interpersonal team relationships.
- It's difficult to have every piece of work critiqued by peers and to have management evaluating and measuring defect density in your code.
- It's extremely important that managers create a culture of collaboration and learning in peer review.
- While it's easy to see defects as purely negative, each bug is actually an opportunity for the team to improve code quality.
- Defects found in peer review are *not* an acceptable rubric by which to evaluate team members.

Other benefits of Code review

- Project awareness
- Peer impression formation
- Consistent coding style
- Design discussion
- Knowledge dissemination